অধ্যায় ২৯

২৯

Advanced JavaScript System Design, Microservices, Distributed Systems এবং Enterprise Architecture

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Functional ও Non-functional Requirement আলাদা করে লিখতে, এবং Capacity Planning-এ সংখ্যা দিয়ে অনুমান করতে (Little's Law-সহ)।
- Monolith, Modular Monolith ও Microservices-এর সুবিধা-অসুবিধা বিচার করে উপযুক্তটি বেছে নিতে।
- Domain-Driven Design-এর Bounded Context দিয়ে Service-এর সীমানা ও Data-র মালিকানা ঠিক করতে।
- Synchronous (HTTP) ও Asynchronous (Event) যোগাযোগের মূল্য বুঝতে।
- একাধিক Service জুড়ে একটি কাজের জন্য Saga Pattern—Orchestration ও Choreography—প্রয়োগ করতে, এবং Compensation লিখতে।
- Idempotency Key ও Request Fingerprint দিয়ে Duplicate ও Conflict সামলাতে।
- CAP-এর আসল অর্থ বুঝে কোথায় দৃঢ় Consistency আর কোথায় Eventual Consistency যথেষ্ট তা ঠিক করতে।
- Timeout, Retry (Backoff + Jitter), Circuit Breaker, Bulkhead, Outbox ও Inbox প্রয়োগ করতে।
- Crash-এর পরে টেকসই Log থেকে অসমাপ্ত কাজ নিরাপদে শেষ করতে।
- **TravelSaga** নামে একটি পরীক্ষিত Booking Orchestrator তৈরি করতে, যেখানে আলাদা HTTP Service, টেকসই Saga Log ও একটি Failure Lab আছে।

পূর্বশর্ত

- অধ্যায় ২৫: Node.js HTTP Server, Idempotency-Key।
- অধ্যায় ২৬: Transaction, UNIQUE Constraint, `node:sqlite`।
- অধ্যায় ২৭: Event Envelope, At-least-once, Transactional Outbox, Backoff ও Jitter।
- অধ্যায় ২৮: Health Check, Structured Log, Metrics ও SLO।

এই অধ্যায় কেন গুরুত্বপূর্ণ

রাফি একটি Cox's Bazar Package-এ দুটি আসন Book করছেন। কাজটি তিন ধাপের: আসন ধরা, টাকা কাটা, Booking নিশ্চিত করা। একটি Application ও একটি Database হলে তিনটিই এক Transaction-এ—সব হবে, নয়তো কিছুই না। কিন্তু প্রতিষ্ঠান বড় হলে আসনের হিসাব রাখে একটি দল ও একটি Service, টাকার হিসাব আরেক দল ও আরেক Service (প্রায়ই একটি বাইরের Payment Gateway), প্রত্যেকের নিজস্ব Database।

এখন নতুন প্রশ্ন: আসন ধরা হলো, কিন্তু টাকা কাটা ব্যর্থ—আসন কে ফেরত দেবে? টাকা কাটার অনুরোধ পাঠানো হলো, কিন্তু উত্তর আসার আগেই Network কেটে গেল—টাকা কি কাটা হয়েছে? আবার চেষ্টা করলে কি দুবার কাটবে? আর Orchestrator ঠিক মাঝখানে বন্ধ হয়ে গেলে, Restart-এর পরে কোথা থেকে শুরু করবে?

Distributed System-এ এই প্রশ্নগুলো ব্যতিক্রম নয়, প্রতিদিনের ঘটনা। এই অধ্যায় সেগুলোর উত্তর দেয় নকশার নীতি দিয়ে, এবং প্রতিটি উত্তর আসল HTTP Service, আসল Timeout ও আসল Crash দিয়ে পরীক্ষা করে।

২৯.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
এক Database-এ Transaction (অধ্যায় ২৬)	একাধিক Service জুড়ে Saga ও Compensation
Idempotency-Key (অধ্যায় ২৫)	Key + Fingerprint; প্রতিটি অংশগ্রহণকারী Service-ও Idempotent
Outbox (অধ্যায় ২৭)	Saga Log, Outbox ও Inbox একসঙ্গে
Backoff ও Jitter (অধ্যায় ২৭)	Timeout, Circuit Breaker, Bulkhead, Retry Amplification
Health ও Metrics (অধ্যায় ২৮)	Capacity Planning, Little's Law
—	Monolith বনাম Microservices, Bounded Context, CAP

> **নোট:** TravelSaga-র Inventory ও Payment দুটি সত্যিকারের আলাদা HTTP Server, প্রত্যেকের নিজস্ব Port ও Data, কিন্তু QA-র সুবিধার জন্য একই Node.js Process-এ চলে। Payment একটি অনুকরণ, আসল Payment Gateway নয়। Orchestrator-এর Saga Log সত্যিকারের SQLite Database (`node:sqlite`), এবং Crash-এর Test আসল Database File ব্যবহার করে। কোনো Message Broker (Kafka, RabbitMQ) ব্যবহার হয়নি; Event-এর প্রবাহ in-memory অনুকরণে দেখানো হয়েছে।

২৯.২ System Design আসলে কী

System Design মানে শুধু Box ও তীর আঁকা নয়। এটি একটি ব্যবসার প্রয়োজনকে একটি নির্ভরযোগ্য, নিরাপদ, রক্ষণযোগ্য, পর্যবেক্ষণযোগ্য এবং খরচ-সচেতন ব্যবস্থায় রূপান্তরের প্রক্রিয়া। শুরু হয় দুই ধরনের Requirement দিয়ে।

Functional Requirement—ব্যবস্থা কী করবে:

- গ্রাহক Package খুঁজতে ও Booking-এর অনুরোধ পাঠাতে পারবেন।
- আসন ধরা হবে; টাকা কাটা সফল হলে Booking নিশ্চিত।
- যেকোনো ধাপ ব্যর্থ হলে আগের ধাপগুলো ফিরিয়ে নেওয়া হবে।
- একই অনুরোধ একাধিকবার এলে একবারই কাজ হবে।

Non-functional Requirement—কতটা ভালোভাবে করবে: Availability, Latency, নিরাপত্তা, টেকসই Data, Scalability, Audit, Disaster Recovery ও খরচ। প্রতিটিকে মাপযোগ্য করুন। "দ্রুত হতে হবে" নয়, "৯৫% অনুরোধ ৩০০ ms-এর মধ্যে"—এবং তা প্রতিশ্রুতি দেওয়ার আগে Traffic-এর ধরন ও প্রতিটি Dependency-র Latency মাপুন। ৩০০ ms কোনো সার্বজনীন মান নয়; একটি বাইরের Payment Gateway নিজেই এক সেকেন্ড নিতে পারে।

২৯.৩ Capacity Planning: অনুমান নয়, হিসাব

capacity.mjs · JavaScript — চলবে: Node.js

```
// Capacity Planning: অনুমান নয়, হিসাব। সংখ্যাগুলো শুরুর অনুমান, পরে মেপে বদলাবে।
const dailyRequests = 86_400;
const peakMultiplier = 20; // সন্ধ্যার Sale-এ গড়ের ২০ গুণ
const latencySeconds = 0.25; // একটি অনুরোধে গড় সময়
const queriesPerRequest = 4;
const cacheHitRate = 0.6;

const avgRps = dailyRequests / (24 * 60 * 60);
const peakRps = avgRps * peakMultiplier;
// Little's Law: একসঙ্গে চলমান অনুরোধ = আগমনের হার x গড় সময়
const concurrent = peakRps * latencySeconds;
const dbQps = peakRps * queriesPerRequest * (1 - cacheHitRate);

console.log({ avgRps, peakRps, concurrent, dbQps });
// Retry Amplification: Payment ব্যর্থ হলে প্রতিটি অনুরোধ ৩ বার চেষ্টা করে
console.log('ব্যর্থতার সময় Payment-এ অনুরোধ/সেকেন্ড:', peakRps * 3);
```

Output

```
{ avgRps: 1, peakRps: 20, concurrent: 5, dbQps: 32 }
ব্যর্থতার সময় Payment-এ অনুরোধ/সেকেন্ড: 60
```

দিনে ৮৬,৪০০ অনুরোধ মানে গড়ে সেকেন্ডে মাত্র একটি—কিন্তু গড় দিয়ে Server কিনবেন না। Peak-এ সেকেন্ডে ২০টি। **Little's Law** বলে: একসঙ্গে চলমান কাজ = আগমনের হার × প্রতিটির গড় সময়; তাই Peak-এ একসঙ্গে ৫টি অনুরোধ চলবে, যা Connection Pool ও Thread-এর মাপ ঠিক করতে কাজে লাগে। Cache ৬০% Query বাঁচালে Database-এ সেকেন্ডে ৩২টি Query। আর শেষ Line একটি সতর্কবার্তা: Payment ব্যর্থ হলে প্রতিটি অনুরোধ তিনবার চেষ্টা করলে, ঠিক যখন Payment দুর্বল, তখনই তার ওপর চাপ তিন গুণ (**Retry Amplification**)।

এগুলো শুরুর অনুমান। Payload-এর আকার, Cache-এর আসল Hit Rate, বাইরের Service-এর Latency—সব মেপে সংখ্যা হালনাগাদ করুন, আর Load Test দিয়ে যাচাই করুন।

২৯.৪ Monolith, Modular Monolith ও Microservices

- **Monolith:** একটি Deploy-যোগ্য Application। Deploy ও Debug সহজ, এক Database-এ Transaction সহজ। কিন্তু ভেতরের সীমানা দুর্বল হলে সবকিছু সবকিছুর সঙ্গে জড়িয়ে যায়।
- **Modular Monolith:** একটিই Deploy, কিন্তু ভেতরে স্পষ্ট Module—প্রতিটির নিজস্ব Data, শুধু Public Interface দিয়ে যোগাযোগ।
- **Microservices:** প্রতিটি Service আলাদাভাবে Deploy ও Scale হয়, প্রায়ই আলাদা দলের মালিকানায়। বিনিময়ে আসে Network-এর ব্যর্থতা, ছড়ানো Data, Distributed Tracing ও অনেক বেশি Operations।

বিষয়	Modular Monolith	Microservices
Deploy	একটি একক	অনেক একক, আলাদা সময়ে
Transaction	এক Database-এ সহজ	Service জুড়ে Saga লাগে
ব্যর্থতা	Function Call ব্যর্থ হয় না	প্রতিটি ডাকে Timeout, Retry
পর্যবেক্ষণ	তুলনামূলক সহজ	Distributed Tracing জরুরি
দলের স্বাধীনতা	মাঝারি	সঠিক সীমানায় বেশি
Operations-এর খরচ	কম	অনেক বেশি

Modular Monolith-এর সীমানা Code-এ কেমন দেখায়:

modular-monolith.mjs · JavaScript — চলবে: Node.js

```
// Modular Monolith: একটি Deploy, কিন্তু প্রতিটি Module শুধু তার Public
// Interface দেয়; অন্য Module-এর Data সরাসরি ছোঁয়া যায় না।
function createInventoryModule(seats) {
  let available = seats; // Private: বাইরে থেকে দেখা বা বদলানো যায় না
  return Object.freeze({
    reserve(count) {
      if (count > available) return { ok: false, reason: 'SOLD_OUT' };
      available -= count;
      return { ok: true };
    },
    release(count) {
      available += count;
    },
    available: () => available,
  });
}

function createBookingModule({ inventory }) {
  const bookings = [];
  return Object.freeze({
    book(customer, seats) {
      const result = inventory.reserve(seats); // শুধু Interface দিয়ে
      if (!result.ok) return result;
      bookings.push({ customer, seats });
      return { ok: true, bookingNo: bookings.length };
    },
  });
}

const inventory = createInventoryModule(3);
const booking = createBookingModule({ inventory });
console.log(booking.book('Rahim', 2));
console.log(booking.book('Mina', 2));
```

```
console.log('বাকি আসন:', inventory.available());
try {
  inventory.available = () => 99; // অন্য Module-এর ভেতরে হাত দেওয়ার চেষ্টা
} catch (error) {
  console.log('আটকানো হলো:', error.constructor.name);
}
```

Output

```
{ ok: true, bookingNo: 1 }
{ ok: false, reason: 'SOLD_OUT' }
বাকি আসন: 1
আটকানো হলো: TypeError
```

Booking Module আসনের সংখ্যা জানে না, শুধু `reserve` ডাকে। `Object.freeze` ও Closure অন্য Module-কে ভেতরে হাত দিতে দেয় না—ES Module Strict Mode-এ চলে বলে চেষ্টা করলে TypeError। এই সীমানা পরিষ্কার থাকলে পরে Inventory-কে আলাদা Service-এ বের করা প্রায় যান্ত্রিক কাজ: Function Call-এর জায়গায় HTTP Call।

পরামর্শ: প্রমাণিত প্রয়োজন—আলাদাভাবে Scale করার চাহিদা, স্বাধীন দল, ভিন্ন Release-এর গতি—না থাকলে Modular Monolith দিয়ে শুরু করুন। Microservices স্বয়ংক্রিয় উন্নতি নয়; এটি একটি মূল্য, যা বিশেষ সমস্যার সমাধানে দেওয়া হয়।

২৯.৫ Domain-Driven Design ও Bounded Context

Bounded Context একটি ব্যবসায়িক এলাকা, যার নিজস্ব ভাষা ও নিয়ম। ভ্রমণ-ব্যবসায়: Catalog, Inventory, Booking, Payment, Notification। একই শব্দের অর্থ Context-ভেদে বদলায়: Booking-এর "নিশ্চিত" আর Payment-এর "Captured" এক নয়; Inventory-র "Reservation" Booking-এর "Booking" নয়। সীমানা টানুন ব্যবসার সক্ষমতা ধরে, Database Table ধরে নয়।

Folder-এর কাঠামো

```
Client / Web App
        |
   API Gateway (Auth, Rate Limit)
        |
 +------+-----------+------------+
 | Catalog      Booking      Identity |
 +-----------------+------------+
                   |
          Saga Orchestrator ---- Saga Log (SQLite)
              /          \
        Inventory       Payment
              \          /
               Event Bus
                   |
              Notification
```

এটি একটি ধারণাগত চিত্র, কোনো Deploy-করা কাঠামো নয়। মূল নিয়ম **Data-র মালিকানা**: প্রতিটি Service শুধু নিজের Database-এ লেখে। Booking Service কখনো Inventory-র Table-এ সরাসরি `UPDATE` চালাবে না—তাহলে Inventory-র নিয়ম (আসন ঋণাত্মক নয়) পাশ কাটানো যায়, আর Inventory দল নিজের Schema বদলাতে পারে না। অন্যের Data চাইলে তার API বা তার প্রকাশিত Event।

২৯.৬ Synchronous বনাম Asynchronous যোগাযোগ

Synchronous (HTTP): অনুরোধ পাঠিয়ে উত্তরের অপেক্ষা। সরল, আর উত্তর সঙ্গে সঙ্গে পাওয়া যায়। কিন্তু ডাকা Service ধীর বা বন্ধ হলে ডাকারীও ধীর বা বন্ধ; একটি অনুরোধের পথে যত Service, Availability তত কম (তিনটি ৯৯.৯% Service পরপর মানে প্রায় ৯৯.৭%)।

Asynchronous (Event): Event প্রকাশ করে এগিয়ে যাওয়া; আগ্রহীরা নিজের সময়ে প্রক্রিয়া করে। Service-গুলো সময়ে বিচ্ছিন্ন, একটি বন্ধ থাকলেও অন্যরা চলে। বিনিময়ে আসে Delivery-র নিশ্চয়তা, ক্রম, Duplicate, Schema-র বিবর্তন, Dead-letter Queue—আর "কাজ শেষ হয়েছে কি না" বোঝা কঠিন হয়।

Event-এর নামে সংস্করণ রাখা (`booking.confirmed.v1`) বা Envelope-এ `version` Field (অধ্যায় ২৭)—যেকোনোভাবে নিয়ম একই: নতুন ঐচ্ছিক Field যোগ নিরাপদ; নাম বা অর্থ বদলানো নতুন সংস্করণ।

২৯.৭ Distributed Transaction ও Saga

দুটি আলাদা Database-এর পরিবর্তন একটি সাধারণ Transaction-এ Commit করা যায় না। Two-Phase Commit (2PC) তাত্ত্বিকভাবে আছে, কিন্তু সব অংশগ্রহণকারী (বিশেষত বাইরের Payment Gateway) তা সমর্থন করে না, এবং সমন্বয়কারী ব্যর্থ হলে সবাই আটকে থাকে। **Saga** ব্যবহারিক বিকল্প: প্রতিটি ধাপ নিজের Service-এ একটি স্থানীয়

Transaction, আর পরের ধাপ ব্যর্থ হলে আগের ধাপগুলোর **Compensation**—একটি যুক্তিসংগত উল্টো কাজ (আসন ছাড়া, Refund)।

Folder-এর কাঠামো

```
সফল পথ:       Booking অনুরোধ -> আসন ধরা -> টাকা কাটা -> নিশ্চিত -> Event
আসন নেই:      আসন ধরা ব্যর্থ (409) -> প্রত্যাখ্যান (টাকায় হাতই পড়েনি)
টাকা ব্যর্থ:  আসন ধরা -> টাকা কাটা ব্যর্থ (402) -> আসন ছাড়া -> প্রত্যাখ্যান
অনিশ্চিত:     আসন ধরা -> টাকা কাটা Timeout -> Refund -> আসন ছাড়া -> প্রত্যাখ্যান
ক্ষতিপূরণ ব্যর্থ:  ... -> আসন ছাড়া ব্যর্থ -> MANUAL_REVIEW (মানুষের হাতে)
```

Compensation সব সময় নিখুঁত "Undo" নয়। পাঠানো Email ফেরানো যায় না, শুধু একটি সংশোধনী পাঠানো যায়; Refund-এ গ্রাহক কয়েক দিন টাকা ছাড়া থাকেন। তাই ধাপের ক্রম সাজান যাতে সহজে ফেরানো যায় এমন কাজ আগে (আসন ধরা), কঠিনটি পরে (টাকা কাটা), আর একেবারে ফেরানো যায় না এমন কাজ (Email) সবশেষে, সব নিশ্চিত হওয়ার পর।

Orchestration বনাম Choreography: Orchestration-এ একটি কেন্দ্রীয় সমন্বয়কারী প্রতিটি ধাপ ডাকে—প্রবাহ এক জায়গায় পড়া ও Debug করা যায়, TravelSaga এটিই করে। Choreography-তে কোনো পরিচালক নেই; প্রতিটি Service Event শুনে নিজের কাজ করে:

choreography.mjs · JavaScript — চলবে: Node.js

```javascript
// Choreography: কোনো কেন্দ্রীয় পরিচালক নেই। প্রতিটি Service Event শুনে
// নিজের কাজ করে এবং নতুন Event ছাড়ে। Compensation-ও Event দিয়ে।
const handlers = new Map();
const log = [];
const on = (type, fn) => handlers.set(type, [...(handlers.get(type) ?? []),
  fn]);
const emit = (type, data) => {
  log.push(type);
  for (const fn of handlers.get(type) ?? []) fn(data);
};

let seats = 5;
const declined = new Set(['Karim']);

on('booking.requested', (b) => (seats >= b.seats
  ? (seats -= b.seats, emit('seats.reserved', b))
  : emit('booking.rejected', { ...b, reason: 'SOLD_OUT' })));
on('seats.reserved', (b) => (declined.has(b.customer)
  ? emit('payment.failed', b) : emit('payment.captured', b)));
on('payment.captured', (b) => emit('booking.confirmed', b));
on('payment.failed', (b) => {
  seats += b.seats; // Inventory নিজেই শুনে আসন ফেরত দেয়
  emit('booking.rejected', { ...b, reason: 'PAYMENT_DECLINED' });
});

for (const customer of ['Rahim', 'Karim']) {
```

```
  log.length = 0;
  emit('booking.requested', { customer, seats: 2 });
  console.log(`${customer}: ${log.join(' > ')}`);
}
console.log('বাকি আসন:', seats);
```

Output

```
Rahim: booking.requested > seats.reserved > payment.captured > booking.confirmed
Karim: booking.requested > seats.reserved > payment.failed > booking.rejected
বাকি আসন: 3
```

Choreography-তে Service-গুলো একে অপরকে চেনে না, শুধু Event চেনে—যোগসূত্র কম। কিন্তু পুরো প্রবাহ কোথাও লেখা নেই; তা ছড়িয়ে আছে বহু Handler-এ। পাঁচ-ছয়টি ধাপের পরে "Booking এখন কোন অবস্থায়, আটকে আছে কোথায়" বোঝা কঠিন হয়। ছোট, স্থির প্রবাহে Choreography; বহু ধাপ, Timeout ও Compensation-যুক্ত ব্যবসায়িক প্রবাহে Orchestration সাধারণত সহজ।

২৯.৮ Idempotency: Key ও Fingerprint

Network Timeout মানে ব্যর্থতা নয়—শুধু "উত্তর জানা নেই"। Client আবার চেষ্টা করবেই, তাই প্রতিটি অবস্থা-বদলানো অনুরোধ Idempotent হতে হবে। দুটি অংশ: **Idempotency Key** বলে "এটি কোন যৌক্তিক অনুরোধ" (এখানে `bookingId`), আর **Fingerprint** বলে "ঠিক কী চাওয়া হয়েছিল"। একই Key ও একই Fingerprint হলে আগের ফল; একই Key কিন্তু ভিন্ন Fingerprint হলে এটি Client-এর Bug—409 Conflict, নীরবে পুরোনো ফল দেওয়া নয়।

fingerprint.mjs · JavaScript — চলবে: Node.js

```
import { fingerprintOf } from '../project/travel-saga/src/orchestrator.mjs';

// একই Idempotency Key, তিনটি অনুরোধ। Fingerprint বলে কোনটি সত্যিই "একই"।
const first = { bookingId: 'trip-7', customer: 'Rahim', seats: 2,
  amount: 1_250_000 };
const reordered = { amount: 1_250_000, seats: 2, customer: 'Rahim',
  bookingId: 'trip-7' };
const changed = { ...first, amount: 125_000 };

const short = (request) => fingerprintOf(request).slice(0, 12);
console.log('প্রথম:      ', short(first));
console.log('ক্রম বদলানো:', short(reordered), '(একই অনুরোধ)');
console.log('দাম বদলানো: ', short(changed), '(Conflict)');
```

Output

```
প্রথম:        f6b104d2fcb3
ক্রম বদলানো: f6b104d2fcb3 (একই অনুরোধ)
দাম বদলানো:  f29354afe76f (Conflict)
```

Fingerprint-এর আগে Key-গুলো সাজানো হয়, তাই JSON-এ Field-এর ক্রম বদলালেও একই অনুরোধ একই Hash পায়। TravelSaga-তে Idempotency দুই স্তরে: Orchestrator `bookingId` ও Fingerprint SQLite-এ রাখে (PRIMARY KEY), আর প্রতিটি অংশগ্রহণকারী Service-ও নিজে Idempotent—Inventory `reservationId`-তে, Payment `chargeId`-তে। এই দ্বিতীয় স্তরই Crash-এর পরে নিরাপদে আবার চালানো সম্ভব করে (অনুচ্ছেদ ২৯.১২)। একাধিক Replica থাকলে Process-এর Memory-র Lock যথেষ্ট নয়; Database-এর UNIQUE Constraint-ই চূড়ান্ত পাহারাদার।

২৯.৯ Consistency, CAP ও Data-র মালিকানা

CAP-কে প্রায়ই "তিনটির যেকোনো দুটি বেছে নিন" বলে সরলীকরণ করা হয়—ভুলভাবে। আসল কথা: Network বিভাজন (**Partition**) চলাকালে একটি ব্যবস্থাকে বেছে নিতে হয়—হয় কিছু অনুরোধ প্রত্যাখ্যান করে Consistency রাখবে (CP), নয়তো সব অনুরোধের উত্তর দিয়ে সাময়িক অসামঞ্জস্য মেনে নেবে (AP)। Partition না থাকলে দুটিই পাওয়া যায় (যদিও Latency-র মূল্যে)।

partition.mjs · JavaScript — চলবে: Node.js

```javascript
// Network Partition চলাকালে দুটি Replica: Consistency না Availability?
// শেষ আসনটি দুই শহরের দুই Agent একই সময়ে বিক্রি করতে চান।
function replica(name, seats) {
  return { name, seats, sold: [] };
}

function sell(r, agent, { partitioned, mode }) {
  if (partitioned && mode === 'CP') {
    return `${r.name}: প্রত্যাখ্যান (অন্য Replica-র সঙ্গে মিলানো যাচ্ছে না)`;
  }
  if (r.seats < 1) return `${r.name}: আসন নেই`;
  r.seats -= 1;
  r.sold.push(agent);
  return `${r.name}: ${agent}-এর কাছে বিক্রি`;
}

for (const mode of ['AP', 'CP']) {
  const dhaka = replica('Dhaka', 1);
  const ctg = replica('Chattogram', 1);
  console.log(`${mode}:`);
  console.log('  ', sell(dhaka, 'Agent-A', { partitioned: true, mode }));
  console.log('  ', sell(ctg, 'Agent-B', { partitioned: true, mode }));
  // Partition সারল; দুই Replica মিলিয়ে দেখা
  const sold = [...dhaka.sold, ...ctg.sold];
```

```
  console.log(`  মিলনের পর বিক্রি: ${sold.length} (আসন ছিল 1)`,
    sold.length > 1 ? 'Overbooking!' : 'ঠিক আছে');
}
```

Output

```
AP:
  Dhaka: Agent-A-এর কাছে বিক্রি
  Chattogram: Agent-B-এর কাছে বিক্রি
  মিলনের পর বিক্রি: 2 (আসন ছিল 1) Overbooking!
CP:
  Dhaka: প্রত্যাখ্যান (অন্য Replica-র সঙ্গে মিলানো যাচ্ছে না)
  Chattogram: প্রত্যাখ্যান (অন্য Replica-র সঙ্গে মিলানো যাচ্ছে না)
  মিলনের পর বিক্রি: ০ (আসন ছিল 1) ঠিক আছে
```

কোনটি সঠিক, তা ব্যবসা ঠিক করে। শেষ আসনের মতো দুর্লভ সম্পদে Overbooking ব্যয়বহুল—তাই Inventory দৃঢ় সমন্বয় চায় (একটি মালিক Database, Transaction, Lock), এমনকি তাতে কিছু অনুরোধ প্রত্যাখ্যাত হলেও। কিন্তু Loyalty Point-এর হিসাব, Analytics বা Notification কয়েক সেকেন্ড দেরিতে হলে কোনো ব্যবসায়িক নিয়ম ভাঙে না—সেখানে **Eventual Consistency** (সময়ের সঙ্গে সব Replica এক অবস্থায় পৌঁছায়) সস্তা ও যথেষ্ট।

২৯.১০ নির্ভরযোগ্যতার Pattern

- **Timeout:** প্রতিটি Network ডাকে। Timeout ছাড়া একটি ধীর Service ধীরে ধীরে ডাকারীর সব সংযোগ ও Memory দখল করে।
- **Retry (Backoff + Jitter):** শুধু সাময়িক ব্যর্থতায় (5xx, Timeout, 429), শুধু Idempotent কাজে, সীমিত সংখ্যায়, আর শুধু একটি স্তরে। 4xx (যেমন Card প্রত্যাখ্যাত) আবার চেষ্টা করলে বদলায় না।
- **Circuit Breaker:** একটি Dependency বারবার ব্যর্থ হলে কিছুক্ষণ তাকে ডাকাই বন্ধ ("খোলা")—দ্রুত ব্যর্থতা, আর অসুস্থ Service-কে সেরে ওঠার সুযোগ। নির্দিষ্ট সময় পরে একটি পরীক্ষামূলক ডাক ("আধা-খোলা"); সফল হলে আবার স্বাভাবিক ("বন্ধ")।
- **Bulkhead:** প্রতিটি Dependency-র জন্য আলাদা সীমিত সম্পদ, যাতে একটির ধীরগতি সবাইকে ডুবিয়ে না দেয় (জাহাজের জলরোধী কক্ষের মতো)।
- **Dead-letter Queue:** বারবার ব্যর্থ Message আলাদা জায়গায়, তদন্ত ও হাতে আবার চালানোর জন্য।
- **Transactional Outbox:** অবস্থা-বদল ও Event একই Transaction-এ (অধ্যায় ২৭)।
- **Inbox:** Consumer-এর দিকে Event-এর ID-তে UNIQUE, যাতে At-least-once Delivery-তে কাজ একবারই।

TravelSaga-র HTTP Client প্রথম তিনটি একসঙ্গে বাস্তবায়ন করে। প্রথম অংশ: Error-এর ধরন ও Circuit Breaker-এর অবস্থা-যন্ত্র:

http-client.mjs · JavaScript অংশ ১ / ৩ (Part 1 of 3) চলবে: Node.js

```
// Service-এর মধ্যে HTTP ডাক: Timeout, সীমিত Retry (Backoff + Jitter)
```

```
// এবং Circuit Breaker। 4xx-এ Retry নয়: সেটি ব্যবসার উত্তর, সাময়িক সমস্যা নয়।
import { setTimeout as delay } from 'node:timers/promises';

export class HttpError extends Error {
  constructor(status, body) {
    super(`HTTP ${status}${body?.error ? ` ${body.error}` : ''}`);
    this.status = status;
    this.body = body;
    this.retryable = status >= 500 || status === 429;
  }
}

export class CircuitOpenError extends Error {
  constructor() {
    super('Circuit open: Dependency-কে বিশ্রাম দেওয়া হচ্ছে');
    this.code = 'CIRCUIT_OPEN';
  }
}

// ...
```

http-client.mjs · JavaScript অংশ ২ / ৩ (Part 2 of 3) চলবে: Node.js

```
// ...

export function createBreaker({ failureThreshold = 3, resetMs = 1000,
  now = Date.now } = {}) {
  let state = 'closed';
  let failures = 0;
  let openedAt = 0;
  return {
    get state() {
      if (state === 'open' && now() - openedAt >= resetMs) state = 'half-open';
      return state;
    },
    before() {
      if (this.state === 'open') throw new CircuitOpenError();
    },
    success() {
      state = 'closed';
      failures = 0;
    },
    failure() {
      failures += 1;
      if (state === 'half-open' || failures >= failureThreshold) {
        state = 'open';
        openedAt = now();
      }
    },
```

```
  };
}

// ...
```

`state` একটি Getter: খোলা অবস্থায় `resetMs` পেরোলে নিজেই "half-open" হয়। Half-open অবস্থায় একটি ব্যর্থতাই আবার খুলে দেয়। এবার ডাক নিজে:

http-client.mjs · JavaScript অংশ ৩ / ৩ (Part 3 of 3) চলবে: Node.js

```
// ...

export const backoffMs = (attempt, random = Math.random, base = 50) =>
  Math.floor(random() * base * 2 ** attempt); // Full Jitter

export async function callJson(url, { method = 'GET', body, timeoutMs = 500,
  retries = 2, breaker, random = Math.random } = {}) {
  for (let attempt = 0; ; attempt += 1) {
    breaker?.before();
    try {
      const res = await fetch(url, { method,
        headers: body ? { 'Content-Type': 'application/json' } : {},
        body: body ? JSON.stringify(body) : undefined,
        signal: AbortSignal.timeout(timeoutMs) });
      const data = await res.json().catch(() => ({}));
      if (res.ok) {
        breaker?.success();
        return { status: res.status, ...data };
      }
      throw new HttpError(res.status, data);
    } catch (error) {
      if (error instanceof HttpError && !error.retryable) {
        breaker?.success(); // Service সুস্থ, শুধু উত্তর "না"
        throw error;
      }
      breaker?.failure();
      if (attempt >= retries) throw error;
      await delay(backoffMs(attempt, random));
    }
  }
}
```

চারটি সিদ্ধান্ত লক্ষ করুন: `AbortSignal.timeout` প্রতিটি চেষ্টাকে সময়সীমা দেয়; 4xx-এ Breaker-এ সাফল্য গোনা হয় (Service সুস্থ, শুধু উত্তর "না") এবং কোনো Retry নেই; Breaker খোলা থাকলে Network-এ যাওয়াই হয় না; আর **Full Jitter**—অপেক্ষা শূন্য থেকে সর্বোচ্চের মধ্যে এলোমেলো—যাতে একসঙ্গে ব্যর্থ হওয়া অনেক Client একসঙ্গে ফিরে না আসে।

২৯.১১ পর্যবেক্ষণ ও নিরাপত্তা

একাধিক Service জুড়ে একটি অনুরোধ Debug করতে **Distributed Trace** লাগে: একটি Trace ID প্রতিটি Service-এ যায় (W3C `traceparent` Header), প্রতিটি Log Line-এ থাকে, এবং OpenTelemetry-র মতো ব্যবস্থা পুরো পথ একটি সময়রেখায় দেখায় (অধ্যায় ২৮)। Saga-র ক্ষেত্রে আরেকটি শক্তিশালী হাতিয়ার হলো Saga Log নিজেই: TravelSaga-র `saga_log` প্রতিটি Booking-এর প্রতিটি অবস্থা-বদল সময়সহ রাখে—"কেন এই Booking প্রত্যাখ্যাত" প্রশ্নের উত্তর সেখানেই।

নিরাপত্তায়: Service থেকে Service-এর ডাকও Authenticate করুন (mTLS বা স্বল্পমেয়াদি Token)—"ভেতরের Network" মানেই বিশ্বস্ত নয়। প্রতিটি Service শুধু যা দরকার তার অনুমতি পায়। Trace ও Log-এ Card নম্বর, Password বা ব্যক্তিগত তথ্য নয়। আর Audit-এর জন্য টাকার প্রতিটি নড়াচড়া একটি অপরিবর্তনীয় Log-এ।

২৯.১২ Project: TravelSaga Booking Orchestrator

TravelSaga একটি Booking-এর তিনটি অংশগ্রহণকারীকে সমন্বয় করে: **Inventory Service** (আসন), **Payment Service** (টাকা) এবং **Orchestrator** (Saga), যার নিজস্ব SQLite Log। দুটি Service সত্যিকারের আলাদা HTTP Server; Orchestrator তাদের সঙ্গে শুধু HTTP-তে কথা বলে, কোনো Function বা Data ভাগ করে না। টাকা Integer পয়সায় (১২,৫০০ টাকা = 1250000), Floating-point-এর গোলমাল এড়াতে।

File-এর কাঠামো

Folder-এর কাঠামো

```
travel-saga/
├── package.json
├── README.md            চালানো, Failure Lab, সীমাবদ্ধতা
├── demo.mjs             সাতটি পরিস্থিতি
├── src/
│   ├── services/
│   │   ├── http-util.mjs   ছোট JSON Server
│   │   ├── inventory.mjs   আসন; reservationId-তে Idempotent
│   │   └── payment.mjs     টাকা (অনুকরণ); chargeId-তে Idempotent
│   ├── http-client.mjs     Timeout, Retry, Circuit Breaker
│   ├── saga-store.mjs      SQLite: অবস্থা, ইতিহাস, Outbox
│   ├── orchestrator.mjs    Saga, Compensation, Resume
│   └── system.mjs          সব একসঙ্গে চালু
└── tests/                  ১৪টি Test
```

অংশগ্রহণকারী Service

Inventory: প্রতিটি কাজ `reservationId` দিয়ে Idempotent। একই ID দ্বিতীয়বার এলে আসন আবার কমে না, আগের ফল (`replay: true`)। আসন ছাড়াও Idempotent: দ্বিতীয়বার `released: false` । `faults` Failure Lab-এর জন্য—পরের কয়েকটি অনুরোধে 503:

inventory.mjs · JavaScript — চলবে: Node.js

```
import { serveJson } from './http-util.mjs';

// Inventory Service: আসনের মালিক। প্রতিটি কাজ reservationId দিয়ে Idempotent,
// তাই Orchestrator নিশ্চিন্তে Retry বা Crash-এর পরে আবার ডাকতে পারে।
export async function startInventory({ seats }) {
  const state = { available: seats, reservations: new Map() };
  const faults = { failNext: 0 };

  function route(method, [resource, id], body) {
    if (method === 'GET' && resource === 'stock') {
      const active = [...state.reservations.values()].filter((r) => r.active);
      return [200, { available: state.available,
        reserved: active.reduce((sum, r) => sum + r.seats, 0) }];
    }
    if (method === 'POST' && resource === 'reservations') {
      if (faults.failNext > 0) {
        faults.failNext -= 1;
        return [503, { error: 'UNAVAILABLE' }];
      }
      const existing = state.reservations.get(body.reservationId);
      if (existing) return [200, { ...existing, replay: true }];
      if (!Number.isInteger(body.seats) || body.seats < 1) {
        return [400, { error: 'INVALID_SEATS' }];
      }
      if (state.available < body.seats) return [409, { error: 'SOLD_OUT' }];
      state.available -= body.seats;
      const reservation = { reservationId: body.reservationId,
        seats: body.seats, active: true };
      state.reservations.set(body.reservationId, reservation);
      return [201, { ...reservation }];
    }
    if (method === 'DELETE' && resource === 'reservations') {
      const reservation = state.reservations.get(id);
      if (!reservation?.active) return [200, { released: false }];
      reservation.active = false;
      state.available += reservation.seats;
      return [200, { released: true }];
    }
    return [404, { error: 'NOT_FOUND' }];
  }

  const server = await serveJson(route);
  return { ...server, state, faults };
}
```

Payment একই নীতিতে, `chargeId` দিয়ে। একই chargeId কিন্তু ভিন্ন Amount এলে 409—আসল Payment Gateway-ও এভাবেই কাজ করে। `delayMs` সবচেয়ে বিপজ্জনক পরিস্থিতি তৈরি করে: টাকা কাটা হয়ে গেছে, কিন্তু উত্তর দেরিতে আসে, তাই Client Timeout পায়:

payment.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
import { serveJson } from './http-util.mjs';

// Payment Service (অনুকরণ, আসল Gateway নয়)। chargeId দিয়ে Idempotent:
// একই chargeId ও একই Amount-এ আগের ফল, ভিন্ন Amount-এ 409।
export async function startPayment() {
  const charges = new Map();
  const faults = { failNext: 0, delayMs: 0, decline: new Set() };

// ...
```

payment.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

  function route(method, [resource, id, action], body) {
    if (method === 'GET' && resource === 'ledger') {
      const all = [...charges.values()];
      const sum = (status) => all.filter((c) => c.status === status)
        .reduce((total, c) => total + c.amount, 0);
      return [200, { captured: sum('CAPTURED'), refunded: sum('REFUNDED'),
        attempts: all.length }];
    }
    if (method === 'POST' && resource === 'charges' && !id) {
      if (faults.failNext > 0) {
        faults.failNext -= 1;
        return [503, { error: 'UNAVAILABLE' }];
      }
      const old = charges.get(body.chargeId);
      if (old && old.amount !== body.amount) {
        return [409, { error: 'IDEMPOTENCY_CONFLICT' }];
      }
      if (!old) {
        const declined = faults.decline.has(body.customer);
        charges.set(body.chargeId, { chargeId: body.chargeId,
          amount: body.amount, status: declined ? 'DECLINED' : 'CAPTURED' });
      }
      const charge = charges.get(body.chargeId);
      // delayMs: টাকা কাটা হয়ে গেছে, কিন্তু উত্তর দেরিতে; Client Timeout পায়।
      if (charge.status === 'DECLINED') return [402, { ...charge }];
      return [old ? 200 : 201, { ...charge }, faults.delayMs];
    }
    if (method === 'POST' && resource === 'charges' && action === 'refund') {
      const charge = charges.get(id);
```

```
      if (charge?.status !== 'CAPTURED') return [200, { refunded: false }];
      charge.status = 'REFUNDED';
      return [200, { refunded: true }];
    }
    return [404, { error: 'NOT_FOUND' }];
  }

  const server = await serveJson(route);
  return { ...server, charges, faults };
}
```

src/saga-store.mjs

Orchestrator-এর টেকসই স্মৃতি। তিনটি Table: `sagas` (বর্তমান অবস্থা ও অনুরোধ), `saga_log` (প্রতিটি অবস্থা-বদলের ইতিহাস), `outbox` (প্রকাশিতব্য Event)। `BEGIN IMMEDIATE` লেখার Lock শুরুতেই নেয়, যাতে দুই সংযোগ একই Saga একসঙ্গে শুরু করতে না পারে:

saga-store.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) — চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

// Orchestrator-এর নিজস্ব টেকসই Log: প্রতিটি Saga-র অবস্থা, ইতিহাস ও Outbox।
// অবস্থা-বদল, ইতিহাস ও Event একই Transaction-এ লেখা হয়।
export const TERMINAL = new Set(['CONFIRMED', 'REJECTED', 'MANUAL_REVIEW']);

export function openSagaStore(path = ':memory:', now = () => new Date()) {
  const db = new DatabaseSync(path);
  db.exec(`CREATE TABLE IF NOT EXISTS sagas (id TEXT PRIMARY KEY,
      fingerprint TEXT NOT NULL, request TEXT NOT NULL,
      status TEXT NOT NULL, reason TEXT, updated_at TEXT NOT NULL);
    CREATE TABLE IF NOT EXISTS saga_log (saga_id TEXT NOT NULL,
      status TEXT NOT NULL, reason TEXT, at TEXT NOT NULL);
    CREATE TABLE IF NOT EXISTS outbox (id INTEGER PRIMARY KEY,
      saga_id TEXT NOT NULL, type TEXT NOT NULL, payload TEXT NOT NULL,
      published INTEGER NOT NULL DEFAULT 0);`);
  const row = (r) => r && { ...r, request: JSON.parse(r.request) };
  const get = (id) => row(db.prepare('SELECT * FROM sagas WHERE id = ?').get(id));

  function inTransaction(fn) {
    db.exec('BEGIN IMMEDIATE');
    try {
      const result = fn();
      db.exec('COMMIT');
      return result;
    } catch (error) {
      db.exec('ROLLBACK');
      throw error;
    }
  }
```

```
// ...
```

`start` নতুন হলে তৈরি করে, আগে থাকলে যা আছে তা-ই ফেরত দেয়—এটিই Idempotency-র ভিত্তি। `transition` অবস্থা, ইতিহাস ও Event—তিনটি একসঙ্গে, একটি Transaction-এ:

saga-store.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

  return {
    get,
    // নতুন হলে তৈরি; আগে থাকলে যা আছে তা-ই (Idempotency-র ভিত্তি)।
    start(id, fingerprint, request) {
      return inTransaction(() => {
        const existing = get(id);
        if (existing) return { created: false, saga: existing };
        const at = now().toISOString();
        db.prepare(`INSERT INTO sagas (id, fingerprint, request, status,
          updated_at) VALUES (?, ?, ?, 'STARTED', ?)`)
          .run(id, fingerprint, JSON.stringify(request), at);
        db.prepare('INSERT INTO saga_log VALUES (?, ?, NULL, ?)')
          .run(id, 'STARTED', at);
        return { created: true, saga: get(id) };
      });
    },
    transition(id, status, reason = null, event = null) {
      inTransaction(() => {
        const at = now().toISOString();
        db.prepare(`UPDATE sagas SET status = ?, reason = ?, updated_at = ?
          WHERE id = ?`).run(status, reason, at, id);
        db.prepare('INSERT INTO saga_log VALUES (?, ?, ?, ?)')
          .run(id, status, reason, at);
        if (event) {
          db.prepare(`INSERT INTO outbox (saga_id, type, payload)
            VALUES (?, ?, ?)`).run(id, event.type, JSON.stringify(event.payload));
        }
      });
    },
    history: (id) => db.prepare(
      'SELECT status, reason FROM saga_log WHERE saga_id = ? ORDER BY rowid')
      .all(id),
    unfinished: () => db.prepare('SELECT * FROM sagas').all().map(row)
      .filter((saga) => !TERMINAL.has(saga.status)),
    pendingEvents: () => db.prepare(
      'SELECT id, saga_id, type FROM outbox WHERE published = 0 ORDER BY id')
      .all(),
    markPublished: (id) => db.prepare(
      'UPDATE outbox SET published = 1 WHERE id = ?').run(id),
```

```
    close: () => db.close(),
  };
}
```

src/orchestrator.mjs

প্রথম অংশ: Validation ও Fingerprint। Validation Saga শুরুর আগেই—ভুল অনুরোধ কোনো Service-কে বিরক্ত করে না:

orchestrator.mjs · JavaScript · অংশ ১ / 8 (Part 1 of 4) · চলবে: Node.js

```
import { createHash } from 'node:crypto';
import { callJson, createBreaker, HttpError } from './http-client.mjs';
import { TERMINAL } from './saga-store.mjs';

export class SagaError extends Error {
  constructor(code, message) {
    super(message);
    this.code = code;
  }
}

function validate(input) {
  const { bookingId, customer, seats, amount } = input ?? {};
  const problems = [];
  if (!/^[\w-]{3,64}$/.test(bookingId ?? '')) problems.push('bookingId');
  if (typeof customer !== 'string' || !customer.trim()) problems.push('customer');
  if (!Number.isInteger(seats) || seats < 1 || seats > 9) problems.push('seats');
  // টাকা পয়সায় (Integer): ১২,৫০০ টাকা = 1250000
  if (!Number.isInteger(amount) || amount <= 0) problems.push('amount');
  if (problems.length) {
    throw new SagaError('VALIDATION', `ভুল Field: ${problems.join(', ')}`);
  }
  return { bookingId, customer: customer.trim(), seats, amount };
}

// একই অনুরোধ চেনার আঙুলের ছাপ: Field-এর ক্রম যা-ই হোক, একই Hash।
export const fingerprintOf = (request) => createHash('sha256')
  .update(JSON.stringify(Object.keys(request).sort()
    .map((key) => [key, request[key]])))
  .digest('hex');

// ...
```

`book`: অনুরোধ Validation, Saga শুরু বা আগেরটি খুঁজে পাওয়া, Fingerprint মিলানো, শেষ হয়ে থাকলে আগের ফল, নইলে কাজ। **Single-flight**: একই Process-এ একই Booking-এর একসঙ্গে আসা পাঁচটি অনুরোধ একটিই কাজ ভাগ করে নেয়:

orchestrator.mjs · JavaScript (অংশ ২ / 8 (Part 2 of 4)) চলবে: Node.js

```
// ...

export function createOrchestrator({ store, inventoryUrl, paymentUrl,
  call = callJson, hooks = {}, callOptions = {} }) {
  const breakers = { inventory: createBreaker(), payment: createBreaker() };
  const inFlight = new Map();
  const inventory = (path, opts) => call(`${inventoryUrl}${path}`,
    { breaker: breakers.inventory, ...callOptions, ...opts });
  const payment = (path, opts) => call(`${paymentUrl}${path}`,
    { breaker: breakers.payment, ...callOptions, ...opts });

  const view = (id) => {
    const saga = store.get(id);
    return { bookingId: id, status: saga.status, reason: saga.reason };
  };

  function finish(id, status, reason, request) {
    const type = { CONFIRMED: 'booking.confirmed', REJECTED: 'booking.rejected',
      MANUAL_REVIEW: 'booking.needs_review' }[status];
    store.transition(id, status, reason, { type, payload: { bookingId: id,
      customer: request.customer, seats: request.seats, reason } });
    return view(id);
  }

  async function book(input) {
    const request = validate(input);
    const id = request.bookingId;
    const { saga } = store.start(id, fingerprintOf(request), request);
    if (saga.fingerprint !== fingerprintOf(request)) {
      throw new SagaError('IDEMPOTENCY_CONFLICT',
        'একই bookingId, কিন্তু ভিন্ন অনুরোধ');
    }
    if (TERMINAL.has(saga.status)) return view(id);
    return singleFlight(id, () => run(id, request));
  }

  // একই Process-এ একই Booking-এর একাধিক ডাক একটিই কাজ ভাগ করে নেয়।
  function singleFlight(id, work) {
    if (!inFlight.has(id)) {
      inFlight.set(id, work().finally(() => inFlight.delete(id)));
    }
    return inFlight.get(id);
  }

// ...
```

Saga-র সামনের পথ। প্রতিটি ধাপের আগে অবস্থা Log-এ লেখা হয়, তাই Crash-এর পরে জানা যায় কোথায় থেমেছিল। ব্যর্থতার ধরন অনুযায়ী আলাদা সিদ্ধান্ত: 409 মানে আসন নেই (কিছু ফেরানোর নেই); 402 মানে প্রত্যাখ্যাত (টাকা কাটা হয়নি, শুধু আসন ফেরত); আর Timeout বা 5xx মানে **অনিশ্চিত**—টাকা কাটা হয়ে থাকতেও পারে, তাই Refund-ও:

orchestrator.mjs · JavaScript অংশ ৩ / ৪ (Part 3 of 4) চলবে: Node.js

```
// ...

  // সামনের প্রতিটি ধাপ Idempotent (একই reservationId ও chargeId), তাই
  // Crash-এর পরে শুরু থেকে আবার চালানো নিরাপদ।
  async function run(id, request) {
    store.transition(id, 'RESERVING');
    try {
      await inventory('/reservations', { method: 'POST',
        body: { reservationId: id, seats: request.seats } });
    } catch (error) {
      if (error instanceof HttpError && error.status === 409) {
        return finish(id, 'REJECTED', 'SOLD_OUT', request);
      }
      // জানা নেই আসন ধরা হয়েছে কি না: ছেড়ে দেওয়ার চেষ্টা (Idempotent)।
      return compensate(id, 'INVENTORY_UNAVAILABLE', request, false);
    }
    await hooks.after?.('reserved', id);
    store.transition(id, 'CHARGING');
    try {
      await payment('/charges', { method: 'POST', body: { chargeId: id,
        amount: request.amount, customer: request.customer } });
    } catch (error) {
      const declined = error instanceof HttpError && error.status === 402;
      // Timeout মানে ব্যর্থতা নয়: টাকা কাটা হয়ে থাকতে পারে, তাই Refund-ও।
      return compensate(id, declined ? 'PAYMENT_DECLINED'
        : 'PAYMENT_UNCERTAIN', request, !declined);
    }
    await hooks.after?.('charged', id);
    return finish(id, 'CONFIRMED', null, request);
  }

// ...
```

Compensation উল্টো ক্রমে, এবং নিজেও ব্যর্থ হতে পারে—তখন `MANUAL_REVIEW`, একটি `booking.needs_review` Event সহ, যাতে একজন মানুষ দেখেন। Error গিলে ফেলা হয় না। শেষে `resume`, Restart-এর পরে:

orchestrator.mjs · JavaScript অংশ 8 / 8 (Part 4 of 4) চলবে: Node.js

```
// ...

  // Compensation উল্টো ক্রমে: আগে Refund, তারপর আসন ছাড়া।
```

```
  async function compensate(id, reason, request, mayBeCharged) {
    store.transition(id, 'COMPENSATING', reason);
    try {
      if (mayBeCharged) {
        await payment(`/charges/${id}/refund`, { method: 'POST' });
      }
      await inventory(`/reservations/${id}`, { method: 'DELETE' });
      return finish(id, 'REJECTED', reason, request);
    } catch (error) {
      // ক্ষতিপূরণও ব্যর্থ: লুকানো নয়, মানুষের হাতে।
      return finish(id, 'MANUAL_REVIEW', `${reason}; ${error.message}`, request);
    }
  }

  // Restart-এর পরে: অসমাপ্ত প্রতিটি Saga শেষ করা।
  async function resume() {
    const results = [];
    for (const saga of store.unfinished()) {
      const work = saga.status === 'COMPENSATING'
        ? () => compensate(saga.id, saga.reason, saga.request, true)
        : () => run(saga.id, saga.request);
      results.push(await singleFlight(saga.id, work));
    }
    return results;
  }

  return { book, resume, breakers, status: view };
}
```

`resume`-এর সরলতা লক্ষ করুন: অসমাপ্ত Saga-কে শুধু শুরু থেকে আবার চালানো। এটি নিরাপদ কারণ প্রতিটি সামনের ধাপ Idempotent—আসন আগে ধরা থাকলে Inventory আগের ফল দেয়, টাকা আগে কাটা থাকলে Payment আগের ফল দেয়। Idempotency না থাকলে Crash-Recovery-র জন্য জটিল "কোন ধাপ সত্যিই হয়েছিল" তদন্ত লাগত।

src/system.mjs ও demo.mjs

`startSystem` দুটি Service আলাদা Port-এ চালু করে, Saga Store খোলে এবং Orchestrator তৈরি করে:

system.mjs · JavaScript — চলবে: Node.js

```
import { startInventory } from './services/inventory.mjs';
import { startPayment } from './services/payment.mjs';
import { openSagaStore } from './saga-store.mjs';
import { createOrchestrator } from './orchestrator.mjs';

// পুরো ব্যবস্থা এক জায়গায় চালু: দুটি আলাদা HTTP Service (প্রত্যেকের নিজস্ব
// Port ও নিজস্ব Data) এবং একটি Orchestrator, যার নিজস্ব SQLite Log।
export async function startSystem({ seats = 10, dbPath = ':memory:',
  hooks, callOptions } = {}) {
  const inventory = await startInventory({ seats });
```

```
  const payment = await startPayment();
  const store = openSagaStore(dbPath);
  const orchestrator = createOrchestrator({ store, hooks, callOptions,
    inventoryUrl: inventory.url, paymentUrl: payment.url });
  const json = (url) => fetch(url).then((res) => res.json());
  return {
    inventory, payment, store, orchestrator,
    stock: () => json(`${inventory.url}/stock`),
    ledger: () => json(`${payment.url}/ledger`),
    async close() {
      await Promise.all([inventory.close(), payment.close()]);
      store.close();
    },
  };
}
```

Demo সাতটি পরিস্থিতি চালায়। Retry-র Jitter-এ স্থির মান (`random: () => 0.5`) দেওয়া, যাতে আউটপুট প্রতিবার এক:

demo.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) — চলবে: Node.js

```
import { startSystem } from './src/system.mjs';

const system = await startSystem({ seats: 10,
  callOptions: { timeoutMs: 150, retries: 1, random: () => 0.5 } });
const { orchestrator, inventory, payment } = system;

async function attempt(label, request) {
  try {
    const result = await orchestrator.book(request);
    console.log(`${label}: ${result.status}${result.reason
      ? ` (${result.reason})` : ''}`);
  } catch (error) {
    console.log(`${label}: ${error.code}`);
  }
}

const trip = { bookingId: 'trip-101', customer: 'Rahim', seats: 2,
  amount: 1_250_000 };
await attempt('১. সাধারণ Booking', trip);
await attempt('২. একই অনুরোধ আবার', trip);
await attempt('৩. একই ID, ভিন্ন দাম', { ...trip, amount: 1 });

// ...
```

demo.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) — চলবে: Node.js

```
// ...
```

```
payment.faults.decline.add('Karim');
await attempt('8. Card প্রত্যাখ্যাত', { bookingId: 'trip-102',
  customer: 'Karim', seats: 3, amount: 900_000 });

inventory.faults.failNext = 1;
await attempt('৫. Inventory সাময়িক 503', { bookingId: 'trip-103',
  customer: 'Mina', seats: 1, amount: 450_000 });

await attempt('৬. আসন নেই', { bookingId: 'trip-104', customer: 'Rafi',
  seats: 8, amount: 100_000 });

payment.faults.delayMs = 300; // টাকা কাটে, কিন্তু উত্তর আসে Timeout-এর পরে
await attempt('৭. Payment-এর উত্তর অনিশ্চিত', { bookingId: 'trip-105',
  customer: 'Nadia', seats: 1, amount: 300_000 });
payment.faults.delayMs = 0;

console.log('Inventory:', await system.stock());
console.log('Payment:', await system.ledger());
console.log('Outbox-এর Event:');
for (const event of system.store.pendingEvents()) {
  console.log(`  ${event.saga_id} ${event.type}`);
}
console.log('trip-105-এর ইতিহাস:', system.store.history('trip-105')
  .map((h) => h.status).join(' > '));
await system.close();
```

Output

```
১. সাধারণ Booking: CONFIRMED
২. একই অনুরোধ আবার: CONFIRMED
৩. একই ID, ভিন্ন দাম: IDEMPOTENCY_CONFLICT
8. Card প্রত্যাখ্যাত: REJECTED (PAYMENT_DECLINED)
৫. Inventory সাময়িক 503: CONFIRMED
৬. আসন নেই: REJECTED (SOLD_OUT)
৭. Payment-এর উত্তর অনিশ্চিত: REJECTED (PAYMENT_UNCERTAIN)
Inventory: { available: 7, reserved: 3 }
Payment: { captured: 1700000, refunded: 300000, attempts: 4 }
Outbox-এর Event:
  trip-101 booking.confirmed
  trip-102 booking.rejected
  trip-103 booking.confirmed
  trip-104 booking.rejected
  trip-105 booking.rejected
trip-105-এর ইতিহাস: STARTED > RESERVING > CHARGING > COMPENSATING > REJECTED
```

হিসাব মিলিয়ে দেখুন। শুধু trip-101 (২ আসন) ও trip-103 (১ আসন) নিশ্চিত, তাই ১০টির ৩টি ধরা। Payment-এ ১২,৫০০ + 8,৫০০ = ১৭,০০০ টাকা কাটা; trip-105-এর ৩,০০০ টাকা কাটা হয়েছিল (অনিশ্চিত উত্তরের পেছনে), তারপর Refund। চারটি Charge-এর চেষ্টা: 101, 102 (প্রত্যাখ্যাত), 103, 105; trip-104 আসন না পেয়ে Payment পর্যন্ত যায়ইনি। প্রতিটি শেষ

হওয়া Saga Outbox-এ একটি Event রেখেছে।

Test: Crash ও Recovery

সবচেয়ে কঠিন পরিস্থিতি: Orchestrator ঠিক টাকা কাটার পরে, কিন্তু "নিশ্চিত" লেখার আগে মারা গেল। Test `hooks.after` দিয়ে ঠিক সেই মুহূর্তে একটি Error ছুড়ে Crash অনুকরণ করে, তারপর একই Database File দিয়ে একটি নতুন Orchestrator তৈরি করে `resume` চালায়:

recovery.test.mjs · JavaScript — চলবে: Node.js

```
// Orchestrator ঠিক ভুল মুহূর্তে মারা গেলে: টেকসই Log থেকে Restart-এর পরে শেষ করা।
import test from 'node:test';
import assert from 'node:assert/strict';
import { mkdtempSync, rmSync } from 'node:fs';
import { tmpdir } from 'node:os';
import { join } from 'node:path';
import { startSystem } from '../src/system.mjs';
import { openSagaStore } from '../src/saga-store.mjs';
import { createOrchestrator } from '../src/orchestrator.mjs';

class Crash extends Error {}

for (const crashPoint of ['reserved', 'charged']) {
  test(`'${crashPoint}'-এর পরে Crash, Restart-এ resume: একবারই কাটা`,
    async (t) => {
      const dir = mkdtempSync(join(tmpdir(), 'saga-'));
      const dbPath = join(dir, 'saga.db');
      const sys = await startSystem({ seats: 5, dbPath,
        hooks: { after: (step) => {
          if (step === crashPoint) throw new Crash(step);
        } } });
      t.after(async () => {
        await sys.close();
        rmSync(dir, { recursive: true, force: true });
      });
      const request = { bookingId: 'trip-9', customer: 'Rahim', seats: 2,
        amount: 800_000 };
      await assert.rejects(sys.orchestrator.book(request), Crash);
      const before = sys.store.get('trip-9').status;
      assert.equal(before, crashPoint === 'reserved' ? 'RESERVING'
        : 'CHARGING');

      // "নতুন Process": একই Database File, নতুন Orchestrator, কোনো Hook নেই।
      const store = openSagaStore(dbPath);
      const restarted = createOrchestrator({ store,
        inventoryUrl: sys.inventory.url, paymentUrl: sys.payment.url });
      const [result] = await restarted.resume();
      assert.equal(result.status, 'CONFIRMED');
      assert.deepEqual(await sys.ledger(),
```

```
        { captured: 800_000, refunded: 0, attempts: 1 });
      assert.deepEqual(await sys.stock(), { available: 3, reserved: 2 });
      assert.deepEqual(await restarted.resume(), []); // আর কিছু বাকি নেই
      store.close();
    });
}
```

দুটি Crash-বিন্দুতেই ফল এক: Booking নিশ্চিত, টাকা ঠিক একবার কাটা (`attempts: 1`), আসন ঠিক একবার ধরা। দ্বিতীয় `resume` কিছুই করে না।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
cd travel-saga
npm test
npm run demo
```

`npm test` ১৪টি Test চালায়: `participants` ২ (দুটি Service-এর Idempotency, Conflict, প্রত্যাখ্যান ও Refund), `http-client` ৪ (Retry, 4xx-এ Retry নয়, Timeout, Circuit Breaker-এর চার অবস্থা), `saga` ৬ (সফল পথ ও Outbox, প্রত্যাখ্যান, আসন নেই, একসঙ্গে পাঁচটি Duplicate ও Conflict, অনিশ্চিত Payment, Compensation ব্যর্থ), এবং `recovery` ২ (দুটি Crash-বিন্দু):

Output

```
# tests 14
# pass 14
# fail 0
```

Test লেখার সময় একটি শিক্ষণীয় ব্যর্থতা: প্রথম চালনায় Saga-র ছয়টি Test-ই "ভুল Field: bookingId" দিয়ে ব্যর্থ হয়েছিল। কারণ Code-এ নয়, Test-এ— `'t1'`-এর মতো ID দুই অক্ষরের, অথচ Validation কমপক্ষে তিন অক্ষর চায়। Validation ঠিক কাজ করছিল; Test-এর ID বদলানো হয়েছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
একই Booking-এ দুবার টাকা	অংশগ্রহণকারী Idempotent নয়, বা প্রতিবার নতুন chargeId	chargeId = bookingId
IDEMPOTENCY_CONFLICT অপ্রত্যাশিত	Client Retry-তে Body বদলাচ্ছে (যেমন সময় যোগ)	Retry-তে হুবহু একই Body
Saga RESERVING বা CHARGING-এ আটকে	Orchestrator মাঝপথে বন্ধ হয়েছিল	চালুর সময় `resume()`

লক্ষণ	সম্ভাব্য কারণ	করণীয়
সব ডাক CIRCUIT_OPEN	Dependency বারবার ব্যর্থ	Dependency-র Health; `resetMs` পরে Half-open
MANUAL_REVIEW জমছে	Compensation ব্যর্থ	`saga_log` দেখে হাতে মিলানো; কারণ ঠিক করা
হঠাৎ Payment-এ অনেক বেশি অনুরোধ	একাধিক স্তরে Retry	Retry এক স্তরে, Breaker

২৯.১৩ Failure Lab

README-র সারণির প্রতিটি পরীক্ষা নিজে চালান এবং ফল ব্যাখ্যা করুন:

১. `payment.faults.decline.add('Karim')` দিয়ে প্রত্যাখ্যান; Inventory-র `stock` আগের মতো কি না দেখুন।
২. আসনের চেয়ে বেশি চেয়ে Sold-out; Payment-এর `ledger`-এ কোনো চেষ্টা নেই তো?
৩. একই Booking দুবার পাঠিয়ে আসনের সংখ্যা অপরিবর্তিত কি না দেখুন।
৪. একই ID কিন্তু ভিন্ন `amount`—IDEMPOTENCY_CONFLICT।
৫. `Promise.all` দিয়ে একসঙ্গে পাঁচটি Duplicate।
৬. Compensation-এর মাঝে Inventory বন্ধ করে MANUAL_REVIEW দেখুন, এবং ভাবুন একজন Operator কী তথ্য পেলে তা হাতে মিলাতে পারবেন।

২৯.১৪ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
Timeout-কে নিশ্চিত ব্যর্থতা ধরে আবার টাকা কাটা	দ্বিগুণ Charge	Idempotent chargeId; অনিশ্চিত হলে Refund বা যাচাই
Microservices মানেই Scalability ভাবা	Network-এর খরচ, জটিলতা	প্রমাণিত প্রয়োজনে, Modular Monolith আগে
অন্য Service-এর Table সরাসরি বদলানো	নিয়ম পাশ কাটানো, Schema আটকে যায়	শুধু API বা Event
At-least-once Event-এ Idempotent নয় এমন Consumer	একই কাজ বারবার	Inbox, UNIQUE
Outbox ছাড়া অবস্থা-বদলের পরে Event প্রকাশ	মাঝে Crash হলে অমিল	একই Transaction-এ Outbox

ভুল	কেন সমস্যা	সমাধান
প্রতিটি স্তরে Retry, Jitter নেই	Retry Storm	এক স্তরে Retry, Backoff + Jitter, Breaker
Compensation ব্যর্থতা গিলে ফেলা	অদৃশ্য অমিল, টাকা আটকে	MANUAL_REVIEW, Alert
Trace Context না পাঠানো	কোন Service-এ দেরি, বোঝা যায় না	`traceparent`, OpenTelemetry
Log-এ Card বা ব্যক্তিগত তথ্য	গোপনীয়তা ভঙ্গ	Redaction, Data কমানো
না মেপে SLA প্রতিশ্রুতি	রাখা যায় না এমন কথা	আগে মাপা, তারপর SLO, তারপর SLA

২৯.১৫ Debugging নির্দেশিকা

১. **Saga Log আগে:** `history(bookingId)`—কোন ধাপ পর্যন্ত গেছে, কোন কারণে থেমেছে।
২. **প্রতিটি অংশগ্রহণকারীর নিজের দৃষ্টি:** Inventory-র Reservation ও Payment-এর Ledger—Saga যা বলছে তার সঙ্গে মিলছে?
৩. **অনিশ্চিত অবস্থা খুঁজুন:** COMPENSATING বা CHARGING-এ পুরোনো Saga মানে Resume চলেনি।
৪. **Trace ID:** একই অনুরোধ কোন Service-এ কত সময় নিল।
৫. **Breaker-এর অবস্থা ও Retry-র সংখ্যা Metrics-এ:** হঠাৎ বাড়লে Dependency অসুস্থ।
৬. **নিয়মিত Reconciliation:** দিনের শেষে Payment Gateway-এর তালিকা ও নিজের Booking মিলিয়ে দেখা—যে অমিল কোনো Alert ধরেনি, তা এখানে ধরা পড়ে।

২৯.১৬ পেশাদার অভ্যাস

- **সীমানা ব্যবসা ধরে, Data-র একজন মালিক।**
- **প্রতিটি Network ডাকে Timeout; Retry শুধু Idempotent কাজে, এক স্তরে।**
- **প্রতিটি অবস্থা-বদলানো API Idempotent, Key + Fingerprint-সহ।**
- **Saga-র প্রতিটি ধাপের আগে টেকসই Log; চালুর সময় Resume।**
- **ফেরানো সহজ কাজ আগে, ফেরানো যায় না এমন কাজ সবশেষে।**
- **Compensation-এর ব্যর্থতা মানুষের কাছে পৌঁছায়।**
- **Architecture-এর সিদ্ধান্ত লিখে রাখুন (ADR):** কী বেছেছেন, কোন বিকল্প, কেন।
- **Architecture Review-এর Checklist:** Requirement ও Traffic-এর অনুমান লিখিত; সীমানা ও Data-র মালিকানা; Auth ও Secret; Idempotency, Retry, Timeout ও Compensation; Event-এর Schema, Outbox ও Inbox; Monitoring, Alert, Trace ও Runbook; Backup, DR ও Reconciliation; Load, Integration, Chaos ও Security Test-এর পরিকল্পনা।

২৯.১৭ অনুশীলনী

প্রাথমিক স্তর

১. দিনে ৪,৩২,০০০ অনুরোধ, Peak গড়ের ১০ গুণ, প্রতিটি ২০০ ms। গড় ও Peak RPS, একসঙ্গে চলমান অনুরোধ, এবং প্রতি Instance ২০টি সামলালে কতগুলো Instance (একটি অতিরিক্তসহ)?

২. Modular Monolith ও Microservices-এর তিনটি Trade-off লিখুন।

৩. Booking, Payment ও Inventory—প্রতিটি Bounded Context কোন Data-র মালিক, আর কোন শব্দ তার নিজস্ব অর্থে ব্যবহার করে?

৪. TravelSaga-তে Card প্রত্যাখ্যাত হলে Saga Log-এ কোন অবস্থাগুলো ক্রমে আসবে? অনিশ্চিত Payment-এ কোনটি আলাদা?

মধ্যম স্তর

৫. Key ও Fingerprint দিয়ে একটি Idempotent Handler লিখুন: একই অনুরোধে আগের ফল, ভিন্ন Body-তে 409।

৬. তিন স্তরে প্রতিটি নিজে তিনবার Retry করলে একটি অনুরোধ নিচের Service-এ কতগুলো ডাক তৈরি করে? সমাধান কী?

৭. `booking.confirmed` Event-এর জন্য একটি Inbox-ভিত্তিক Consumer লিখুন, যা Loyalty Point একবারই যোগ করে।

৮. Inventory কেন দৃঢ় সমন্বয় চায়, অথচ Notification Eventual Consistency মেনে নিতে পারে? একটি করে ব্যর্থতার উদাহরণ দিন।

চ্যালেঞ্জ স্তর

৯. উদাহরণ ০৩-এর Choreography-তে "Payment অনিশ্চিত" পথ যোগ করার নকশা করুন। Orchestration-এর তুলনায় কী কঠিন হলো?

১০. একটি Bulkhead লিখুন: প্রতি Dependency-তে সর্বোচ্চ তিনটি একসঙ্গে ডাক, বাকিরা সঙ্গে সঙ্গে প্রত্যাখ্যাত।

১১. TravelSaga-র Outbox-এর জন্য একটি Relay লিখুন: ক্রম রক্ষা করে প্রকাশ, ব্যর্থ হলে পরের দফায়।

১২. TravelSaga-কে Production-এ নেওয়ার একটি Architecture প্রস্তাব লিখুন।

২৯.১৮ অনুশীলনীর সমাধান

সমাধান ১

capacity-answer.mjs · JavaScript — চলবে: Node.js

```
// ৪,৩২,০০০ দৈনিক অনুরোধ; Peak গড়ের ১০ গুণ; প্রতি অনুরোধ ২০০ ms।
const daily = 432_000;
const avg = daily / 86_400;
const peak = avg * 10;
const concurrent = peak * 0.2; // Little's Law

console.log(`গড়: ${avg} RPS, Peak: ${peak} RPS`);
console.log(`একসঙ্গে চলমান অনুরোধ (Peak): ${concurrent}`);
// প্রতি Instance নিরাপদে ২০টি একসঙ্গে সামলায়; একটি নষ্ট হলেও চলতে হবে (N+1)
const perInstance = 20;
const instances = Math.ceil(concurrent / perInstance) + 1;
console.log(`Instance: ${instances} (একটি অতিরিক্ত, ব্যর্থতার জন্য)`);
```

Output

```
গড়: 5 RPS, Peak: 50 RPS
একসঙ্গে চলমান অনুরোধ (Peak): 10
Instance: 2 (একটি অতিরিক্ত, ব্যর্থতার জন্য)
```

একসঙ্গে ১০টি অনুরোধ একটি Instance-ই সামলাতে পারে (২০ সীমা), কিন্তু একটি নষ্ট হলে সেবা বন্ধ—তাই N+1 নিয়মে দুটি। Deploy-এর সময় একটি বন্ধ থাকে, সেটিও মনে রাখুন।

সমাধান ২ — (১) **Deploy:** Microservices-এ প্রতিটি দল স্বাধীনভাবে Release করে; বিনিময়ে অনেক Pipeline, সংস্করণের সামঞ্জস্য ও Operations। (২) **Scaling:** শুধু ব্যস্ত অংশটি (যেমন Search) আলাদাভাবে বাড়ানো যায়; বিনিময়ে প্রতিটি ডাক Network-এ—Latency ও ব্যর্থতার নতুন উৎস। (৩) **Data:** Modular Monolith-এ এক Transaction-এ একাধিক Module-এর পরিবর্তন সহজ; Microservices-এ Saga, Compensation ও Eventual Consistency।

সমাধান ৩

Context	মালিকানার Data	নিজস্ব ভাষা
Booking	Booking, তার অবস্থা ও ইতিহাস	"নিশ্চিত", "প্রত্যাখ্যাত", "অনুরোধ"
Payment	Charge, Refund, Ledger	"Captured", "Declined", "Refunded"
Inventory	আসন, Reservation	"Available", "Reserved", "Released"

Booking-এর "নিশ্চিত" ঘটে কেবল Payment-এর "Captured" ও Inventory-র "Reserved"—দুটির পরে; তিনটি শব্দ তিন Context-এর, এবং কোনোটিই অন্যের Table-এ লেখা হয় না।

সমাধান ৪ — প্রত্যাখ্যানে: STARTED, RESERVING, CHARGING, COMPENSATING (কারণ PAYMENT_DECLINED), REJECTED। অনিশ্চিত Payment-এ অবস্থার ক্রম একই, কিন্তু কারণ PAYMENT_UNCERTAIN, এবং COMPENSATING-এর সময় আসন ছাড়ার আগে একটি Refund-ও পাঠানো হয়—কারণ টাকা কাটা হয়ে থাকতে পারে। প্রত্যাখ্যানে Refund পাঠানো হয় না, কারণ 402 নিশ্চিত করে টাকা কাটা হয়নি (`compensate`-এর `mayBeCharged`)।

সমাধান ৫

idempotency.mjs · JavaScript — চলবে: Node.js

```
import { createHash } from 'node:crypto';

// Key ও Fingerprint একসঙ্গে: Key বলে "কোন অনুরোধ", Fingerprint বলে "কী চাওয়া"।
const sha = (value) => createHash('sha256')
  .update(JSON.stringify(value)).digest('hex');
const saved = new Map(); // আসলে: Database Table, Key-তে UNIQUE

function handle(key, body, work) {
  const fingerprint = sha(body);
  const old = saved.get(key);
  if (old && old.fingerprint !== fingerprint) {
    return { status: 409, body: { error: 'IDEMPOTENCY_CONFLICT' } };
  }
  if (old) return { ...old.response, replay: true };
  const response = work(body);
  saved.set(key, { fingerprint, response });
  return response;
}

let charges = 0;
const charge = (body) => {
  charges += 1;
  return { status: 201, body: { charged: body.amount } };
};

console.log(handle('k-1', { amount: 500 }, charge));
console.log(handle('k-1', { amount: 500 }, charge));
console.log(handle('k-1', { amount: 900 }, charge));
console.log('আসলে টাকা কাটা হয়েছে:', charges, 'বার');
```

Output

```
{ status: 201, body: { charged: 500 } }
{ status: 201, body: { charged: 500 }, replay: true }
{ status: 409, body: { error: 'IDEMPOTENCY_CONFLICT' } }
আসলে টাকা কাটা হয়েছে: 1 বার
```

Production-এ `saved` একটি Database Table, Key-তে UNIQUE, আর কাজ ও সারি একই Transaction-এ; নইলে দুটি Replica একই মুহূর্তে "আগে নেই" দেখে দুজনেই কাজ করবে।

সমাধান ৬

retry-storm.mjs · JavaScript — চলবে: Node.js

```javascript
// তিন স্তর (Gateway, Booking, Payment Client), প্রতিটি নিজে ৩ বার চেষ্টা করে।
// নিচের Service পুরো বন্ধ থাকলে একটি ব্যবহারকারী-অনুরোধ কতগুলো ডাক তৈরি করে?
function callsToBottom(layers, attemptsPerLayer) {
  return attemptsPerLayer ** layers;
}

for (const attempts of [1, 2, 3]) {
  const calls = callsToBottom(3, attempts);
  console.log(`প্রতি স্তরে ${attempts} চেষ্টা: ${calls}টি ডাক`);
}
// সমাধান: Retry শুধু এক স্তরে (সাধারণত নিচের Client-এ), বাকিরা দ্রুত ব্যর্থ।
console.log('শুধু নিচের স্তরে ৩ চেষ্টা:', 1 * 1 * 3, 'টি ডাক');
// আর Circuit Breaker খুললে পরের অনুরোধগুলো নিচে পৌঁছায়ই না।
const peakRps = 200;
console.log('৩^৩ হলে বন্ধ Service-এ অনুরোধ/সেকেন্ড:', peakRps * 27);
```

Output

```
প্রতি স্তরে 1 চেষ্টা: 1টি ডাক
প্রতি স্তরে 2 চেষ্টা: ৪টি ডাক
প্রতি স্তরে 3 চেষ্টা: 27টি ডাক
শুধু নিচের স্তরে ৩ চেষ্টা: 3 টি ডাক
৩^৩ হলে বন্ধ Service-এ অনুরোধ/সেকেন্ড: 5400
```

গুণফল সূচকীয়: সেকেন্ডে ২০০ অনুরোধ একটি অসুস্থ Service-এ ৫,৪০০ হয়ে আঘাত করে—সেরে ওঠার কোনো সুযোগ নেই। সমাধান: Retry শুধু এক স্তরে; ওপরের স্তরগুলো দ্রুত ব্যর্থ; Circuit Breaker; এবং একটি **Retry Budget** (মোট অনুরোধের, যেমন, ১০%-এর বেশি Retry নয়)।

সমাধান ৭

inbox.mjs · JavaScript — চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

// Inbox: Consumer-এর দিকে Deduplication। Event-এর ID-তে PRIMARY KEY, আর
// কাজ ও Inbox-এর সারি একই Transaction-এ; তাই Duplicate কখনো দুবার নয়।
const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE inbox (event_id TEXT PRIMARY KEY);
  CREATE TABLE loyalty (customer TEXT PRIMARY KEY, points INTEGER);
  INSERT INTO loyalty VALUES ('Rahim', 0);`);

function onBookingConfirmed(event) {
  db.exec('BEGIN');
  try {
    db.prepare('INSERT INTO inbox VALUES (?)').run(event.id);
    db.prepare('UPDATE loyalty SET points = points + ? WHERE customer = ?')
      .run(event.seats * 100, event.customer);
    db.exec('COMMIT');
    return 'processed';
  } catch (error) {
    db.exec('ROLLBACK');
    if (error.code === 'ERR_SQLITE_ERROR' && /UNIQUE/.test(error.message)) {
      return 'duplicate';
    }
    throw error;
  }
}

const event = { id: 'evt-trip-101', customer: 'Rahim', seats: 2 };
for (const delivery of [1, 2, 3]) {
  console.log(`Delivery ${delivery}:`, onBookingConfirmed(event));
}
console.log({ ...db.prepare('SELECT * FROM loyalty').get() });
```

Output

```
Delivery 1: processed
Delivery 2: duplicate
Delivery 3: duplicate
{ customer: 'Rahim', points: 200 }
```

Inbox-এর সারি ও Point যোগ একই Transaction-এ: দ্বিতীয় Delivery-তে PRIMARY KEY ভঙ্গ হয়, Transaction Rollback হয়, Point দ্বিতীয়বার যোগ হয় না। শুধু "আগে দেখেছি কি" যাচাই করে তারপর আলাদাভাবে লিখলে দুটি Consumer-এর মাঝে Race থাকত।

সমাধান ৮ — Inventory: শেষ আসন দুজনের কাছে বিক্রি হলে একজন গ্রাহক বিমানবন্দরে গিয়ে আসন পান না—টাকা ফেরত, ক্ষতিপূরণ, সুনামহানি (উদাহরণ ০৪-এর AP Overbooking)। তাই এক মালিক, Transaction বা Lock, এবং Partition-এ প্রত্যাখ্যান মেনে নেওয়া। Notification: নিশ্চিতকরণের SMS দশ সেকেন্ড বা এক মিনিট দেরিতে গেলে কোনো ব্যবসায়িক নিয়ম ভাঙে না; ব্যর্থ হলে Retry করা যায়। তাই At-least-once Event ও Idempotent Consumer যথেষ্ট—দৃঢ় সমন্বয়ের খরচ অপ্রয়োজনীয়।

সমাধান ৯ — Payment Service অনিশ্চিত উত্তরে `payment.uncertain` Event ছাড়বে। কিন্তু কে Refund করবে? Payment নিজেই (সে জানে কী কাটা হয়েছে), তারপর `payment.refunded`; Inventory শুনবে `payment.refunded` ও `payment.failed` দুটিই, এবং আসন ছেড়ে `seats.released`; Booking শুনবে `seats.released` এবং প্রত্যাখ্যান লিখবে। কঠিন যা হলো: (১) প্রবাহ এখন চারটি Handler-এ ছড়ানো, কোথাও এক জায়গায় লেখা নেই; (২) "Refund-ও ব্যর্থ হলে কী" প্রশ্নের মালিক কেউ নয়; (৩) একটি Event হারালে Booking চিরকাল অসমাপ্ত, আর তা ধরতে আলাদা Timeout-পর্যবেক্ষক লাগে। Orchestrator-এ এই তিনটিই এক Function-এ (`compensate`) ও এক Log-এ।

সমাধান ১০

bulkhead.mjs · JavaScript — চলবে: Node.js

```
// Bulkhead: প্রতিটি Dependency-র জন্য আলাদা সীমিত "কক্ষ"। ধীর Payment
// সব সংযোগ দখল করলেও Inventory-র ডাক আটকে যায় না।
function createBulkhead(limit) {
  let active = 0;
  let rejected = 0;
  return {
    async run(task) {
      if (active >= limit) {
        rejected += 1;
        throw new Error('BULKHEAD_FULL');
      }
      active += 1;
      try {
        return await task();
      } finally {
        active -= 1;
      }
    },
    stats: () => ({ active, rejected }),
  };
}

const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
const payment = createBulkhead(3);
const inventory = createBulkhead(3);
```

```
// Payment ধীর: ১০টি ডাক একসঙ্গে, প্রতিটি ১০০ ms।
const slow = Array.from({ length: 10 }, () =>
  payment.run(() => wait(100)).catch((e) => e.message));
const fast = await inventory.run(async () => 'Inventory: উত্তর এল');
console.log(fast);
const results = await Promise.all(slow);
console.log('Payment:', results.filter((r) => r === 'BULKHEAD_FULL').length,
  'টি সঙ্গে সঙ্গে প্রত্যাখ্যাত,', results.filter((r) => r === undefined)
    .length, 'টি সম্পন্ন');
console.log(payment.stats());
```

Output

```
Inventory: উত্তর এল
Payment: 7 টি সঙ্গে সঙ্গে প্রত্যাখ্যাত, 3 টি সম্পন্ন
{ active: 0, rejected: 7 }
```

ধীর Payment-এর দশটি ডাকের তিনটি চলে, সাতটি সঙ্গে সঙ্গে প্রত্যাখ্যাত (দীর্ঘ অপেক্ষার বদলে দ্রুত ব্যর্থতা), আর Inventory-র ডাক নিজের কক্ষে বাধাহীন। বাস্তবে প্রত্যাখ্যানের বদলে একটি ছোট, সীমিত Queue-ও রাখা যায়।

সমাধান ১১

outbox-relay.mjs · JavaScript — চলবে: Node.js

```
import { openSagaStore } from '../../project/travel-saga/src/saga-store.mjs';

// Relay: Outbox থেকে Event নিয়ে Broker-এ; সফল হলে তবেই published।
async function relayOnce(store, publish) {
  let sent = 0;
  for (const event of store.pendingEvents()) {
    try {
      await publish(event);
      store.markPublished(event.id);
      sent += 1;
    } catch {
      break; // ক্রম রক্ষা: ব্যর্থ হলে পরেরগুলো পরের দফায়
    }
  }
  return sent;
}

const store = openSagaStore();
const request = { bookingId: 'trip-1', customer: 'Rahim', seats: 2,
  amount: 100 };
for (const id of ['trip-1', 'trip-2', 'trip-3']) {
  store.start(id, 'fp', { ...request, bookingId: id });
  store.transition(id, 'CONFIRMED', null, { type: 'booking.confirmed',
    payload: { bookingId: id } });
```

```
}

let brokerDown = true;
const broker = [];
const publish = async (event) => {
  if (brokerDown && event.saga_id === 'trip-2') throw new Error('down');
  broker.push(event.saga_id);
};
console.log('দফা ১:', await relayOnce(store, publish), 'টি পাঠানো');
brokerDown = false;
console.log('দফা ২:', await relayOnce(store, publish), 'টি পাঠানো');
console.log('Broker পেল:', broker.join(', '));
console.log('বাকি:', store.pendingEvents().length);
```

Output

```
দফা ১: 1 টি পাঠানো
দফা ২: 2 টি পাঠানো
Broker পেল: trip-1, trip-2, trip-3
বাকি: 0
```

প্রথম দফায় trip-2-এ ব্যর্থতা, তাই trip-3-ও অপেক্ষা করে—ক্রম রক্ষা। এর মূল্য: একটি "বিষাক্ত" Event পুরো Queue আটকে দিতে পারে; তাই কয়েকবার ব্যর্থতার পর Dead-letter-এ সরানো (অধ্যায় ২৭-এর সমাধান ৯)। ক্রম যদি শুধু একই Booking-এর মধ্যে জরুরি হয়, তবে প্রতিটি `saga_id` আলাদা Queue হিসেবে চালানো যায়।

সমাধান ১২ — (১) **Process:** Inventory, Payment-Adapter ও Orchestrator আলাদা Container, প্রত্যেকের নিজস্ব Database (PostgreSQL)। (২) **Payment:** আসল Gateway-এর Idempotency-Key ব্যবহার; অনিশ্চিত উত্তরে আগে Gateway-কে জিজ্ঞাসা ("এই Key-র Charge আছে?"), তারপর সিদ্ধান্ত। (৩) **Saga Log:** PostgreSQL-এ; একাধিক Orchestrator-এর মধ্যে একই Saga দুবার না চলে তাই Row Lock বা Lease (`locked_until`)। (৪) **Resume:** একটি Background Worker নিয়মিত পুরোনো অসমাপ্ত Saga খোঁজে। (৫) **Outbox Relay:** আলাদা Worker, Broker-এ (যেমন RabbitMQ বা Kafka) প্রকাশ; Consumer-রা Inbox-সহ। (৬) **Reconciliation:** প্রতিদিন Gateway-এর Settlement File ও নিজের Ledger মিলানো। (৭) **পর্যবেক্ষণ:** প্রতিটি অবস্থায় Saga-র সংখ্যা, অবস্থায় কাটানো সময়, MANUAL_REVIEW-এর হার—Alert-সহ; Trace ID সব Service-এ। (৮) **নিরাপত্তা:** Service-এর মধ্যে mTLS, Payment-এর Secret Secret Store-এ, Card-এর তথ্য কখনো নিজের ব্যবস্থায় নয় (Gateway-এর Token)।

২৯.১৯ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. Saga কী?

ক) একটি CSS Library

খ) স্থানীয় Transaction-এর ধারা, ব্যর্থতায় Compensation-সহ

গ) একটি Encryption পদ্ধতি

ঘ) Browser-এর Storage

প্রশ্ন ২. একই Idempotency Key কিন্তু ভিন্ন Body এলে সঠিক উত্তর কী?

ক) নতুন কাজ করা

খ) আগের ফল নীরবে ফেরত দেওয়া

গ) 409 Conflict

ঘ) 500 Error

প্রশ্ন ৩. Payment-এর ডাকে Timeout হলে TravelSaga কী ধরে নেয়?

ক) টাকা কাটা হয়নি

খ) টাকা কাটা হয়েছে, Booking নিশ্চিত

গ) কিছুই না, চুপচাপ অপেক্ষা

ঘ) অনিশ্চিত: Refund ও আসন ছাড়া

প্রশ্ন ৪. Circuit Breaker খোলা থাকলে কী ঘটে?

ক) ডাক Network-এ না গিয়েই দ্রুত ব্যর্থ হয়

খ) ডাক দ্বিগুণ দ্রুত যায়

গ) Service Restart হয়

ঘ) Retry-এর সংখ্যা বাড়ে

প্রশ্ন ৫. কোন HTTP উত্তরে Retry করা উচিত নয়?

ক) 503

খ) 402 (Card প্রত্যাখ্যাত)

গ) Timeout

ঘ) 429

প্রশ্ন ৬. Network Partition চলাকালে শেষ আসন বিক্রির জন্য সাধারণত কোনটি নিরাপদ?

ক) দুই Replica-ই বিক্রি করুক (AP)

খ) পরে দেখা যাবে

গ) সমন্বয় না হলে প্রত্যাখ্যান (CP)

ঘ) এলোমেলো একটি

প্রশ্ন ৭. TravelSaga-তে Crash-এর পরে অসমাপ্ত Saga শুরু থেকে আবার চালানো নিরাপদ কেন?

ক) অংশগ্রহণকারী Service-গুলো reservationId ও chargeId দিয়ে Idempotent

খ) Crash কখনো ঘটে না

গ) SQLite নিজেই Undo করে

ঘ) Payment সব সময় ব্যর্থ হয়

প্রশ্ন ৮. তিন স্তরের প্রতিটি নিজে তিনবার চেষ্টা করলে নিচের Service-এ একটি অনুরোধ কতগুলো ডাক হতে পারে?

ক) ৩

খ) ৯

গ) ৬

ঘ) ২৭

প্রশ্ন ৯. Bounded Context কী নির্দেশ করে?

ক) একটি TCP Port

খ) একটি ব্যবসায়িক এলাকা, তার নিজস্ব ভাষা, নিয়ম ও Data

গ) একটি Git Branch

ঘ) Memory-র আকার

প্রশ্ন ১০. Compensation-ও ব্যর্থ হলে সঠিক আচরণ কী?

ক) Error লুকিয়ে রাখা

খ) Booking নিশ্চিত দেখানো

গ) MANUAL_REVIEW-এ রেখে মানুষকে জানানো

ঘ) পুরো Database মুছে ফেলা

২৯.২০ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ঘ · ৪–ক · ৫–খ · ৬–গ · ৭–ক · ৮–ঘ · ৯–খ · ১০–গ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	প্রতিটি ধাপ নিজের Service-এ; ব্যর্থতায় আগের ধাপের যুক্তিসংগত উল্টো কাজ।
২	গ	একই Key-তে ভিন্ন চাওয়া Client-এর Bug; উদাহরণ ০৫ ও সমাধান ৫।
৩	ঘ	Timeout মানে "জানা নেই"; Demo-র trip-105-এ টাকা কাটা হয়েছিল, পরে Refund।
৪	ক	Test-এ খোলা অবস্থায় Service-এ ডাকের সংখ্যা বাড়েনি।
৫	খ	প্রত্যাখ্যান একটি ব্যবসায়িক উত্তর; আবার চেষ্টায় বদলায় না।
৬	গ	উদাহরণ ০৪: AP-তে Overbooking, CP-তে কেউ পায়নি কিন্তু ভুলও হয়নি।
৭	ক	আগে হয়ে থাকা ধাপ আবার ডাকলে আগের ফল আসে; Recovery Test-এ attempts ১।
৮	ঘ	৩ × ৩ × ৩ = ২৭; সমাধান ৬।
৯	খ	যেমন Payment-এর "Captured" আর Booking-এর "নিশ্চিত" আলাদা ধারণা।
১০	গ	`booking.needs_review` Event ও Saga Log মানুষকে তথ্য দেয়।

২৯.২১ অধ্যায়ের সারসংক্ষেপ

- System Design শুরু হয় Functional ও মাপযোগ্য Non-functional Requirement দিয়ে; Capacity-র হিসাবে Peak ও Little's Law।
- Modular Monolith প্রায়ই সঠিক শুরু; Microservices একটি মূল্য, যা নির্দিষ্ট প্রয়োজনে দেওয়া হয়।
- Bounded Context ব্যবসা ধরে; প্রতিটি Data-র একজন মালিক, অন্যরা শুধু API বা Event দিয়ে।
- Synchronous সরল কিন্তু যুক্ত; Asynchronous বিচ্ছিন্ন কিন্তু জটিল।
- Saga: স্থানীয় ধাপ ও Compensation; ফেরানো সহজ কাজ আগে; Orchestration বনাম Choreography।
- Idempotency: Key + Fingerprint, এবং প্রতিটি অংশগ্রহণকারী Idempotent—তাতেই Crash-Recovery সরল।
- Timeout মানে অনিশ্চিত, ব্যর্থতা নয়।
- CAP: Partition-এ Consistency বা Availability; দুর্লভ সম্পদে দৃঢ় সমন্বয়, বাকিতে Eventual।
- Timeout, এক-স্তরের Retry, Circuit Breaker, Bulkhead, Outbox, Inbox, Dead-letter।
- টেকসই Saga Log, Resume, MANUAL_REVIEW ও Reconciliation।

পরের ও শেষ অধ্যায়ে এই বইয়ের সব দক্ষতা এক জায়গায় আসবে: একটি পূর্ণাঙ্গ Full-Stack Capstone—TravelMaster Pro—যেখানে Authentication, Validation, Database Transaction, Idempotency, Test, Browser Client ও Deployment-এর প্রস্তুতি একসঙ্গে, আর তার সঙ্গে একটি পেশাদার Career Roadmap।

২৯.২২ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Functional / Non-functional Requirement	কী করবে / কতটা ভালোভাবে করবে।
Little's Law	একসঙ্গে চলমান = আগমনের হার x গড় সময়।
Modular Monolith / Microservices	একটি Deploy, ভেতরে স্পষ্ট Module-সীমানা / আলাদাভাবে Deploy ও Scale-যোগ্য Service।
Bounded Context	নিজস্ব ভাষা ও নিয়মের ব্যবসায়িক এলাকা।
Saga / Compensation	স্থানীয় Transaction-এর ধারা / আগের ধাপের যুক্তিসংগত উল্টো কাজ।
Orchestration / Choreography	কেন্দ্রীয় সমন্বয় / Event-নির্ভর স্বতন্ত্র প্রতিক্রিয়া।
Fingerprint	অনুরোধের বিষয়বস্তুর Hash।
CAP / Eventual Consistency	Partition-এ Consistency ও Availability-র মধ্যে বেছে নেওয়া / সময়ের সঙ্গে সব Replica এক অবস্থায় পৌঁছানো।
Circuit Breaker / Bulkhead	বারবার ব্যর্থ Dependency-কে সাময়িক বিশ্রাম / প্রতিটি Dependency-র জন্য আলাদা সীমিত সম্পদ।
Inbox	Consumer-এর দিকে Event-এর Deduplication।
Reconciliation	দুটি ব্যবস্থার হিসাব নিয়মিত মিলিয়ে দেখা।

২৯.২৩ আরও পড়ুন

- **Microservices.io — Saga pattern:** `https://microservices.io/patterns/data/saga.html` — Orchestration ও Choreography।
- **Microsoft Azure Architecture Center — Saga:** `https://learn.microsoft.com/en-us/azure/architecture/patterns/saga` — Compensation-এর নকশা।
- **Martin Fowler — Microservices:** `https://martinfowler.com/articles/microservices.html` — মূল ধারণা ও মূল্য।
- **Martin Fowler — MonolithFirst:** `https://martinfowler.com/bliki/MonolithFirst.html` — কেন Monolith দিয়ে শুরু।
- **AWS Builders' Library:** `https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/` ও `https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/` — Timeout, Jitter ও নিরাপদ Retry।
- **Stripe — Idempotent requests:** `https://docs.stripe.com/api/idempotent_requests` — একটি আসল Payment API-র Idempotency।